

O jogo ganha vida

input(), random, e o laço que junta tudo o que já foi construído.

Nada aqui é mais difícil do que o que vocês já fizeram — é o mesmo tipo de peça pequena, uma de cada vez, só que agora encaixando em cima do `atacar()`, `defender()` e `usar_pocao()` que já existem nas suas classes. No fim do dia, as peças se juntam e o jogo passa a rodar sozinho, de verdade, do começo ao fim.

O que é novo hoje

Duas ferramentas, com exemplos, antes dos exercícios.

input()

```
escolha = input("Escolha seu personagem (1-Guerreiro / 2-Mago): ")
```

`input()` sempre devolve uma string, mesmo se a pessoa digitar um número. Pra comparar com números de verdade, converta com `int()`:

```
escolha = int(input("Escolha seu personagem (1-Guerreiro / 2-Mago): "))

if escolha == 1:
    print("Guerreiro escolhido.")
```

Sem o `int()`, `escolha` guarda o texto "1", não o número 1, e `"1" == 1` é `False` em Python. Quando `input()` for usado pra comparar com **texto** (tipo "atacar"), não precisa de `int()` nenhum — só quando for comparar com número.

random

```
import random

dano = random.randint(5, 10)
```

`random.randint(a, b)` sorteia um número inteiro entre `a` e `b`, incluindo os dois extremos. Precisa de `import random` no topo do arquivo, antes de qualquer outra coisa.

Um exemplo clássico de RPG de mesa — simular a rolagem de um dado de 20 lados:

```
rolagem = random.randint(1, 20)
print("Voce rolou um d20:", rolagem)

if rolagem >= 15:
    print("Acerto critico!")
elif rolagem >= 8:
    print("Acertou.")
else:
    print("Errou o golpe.")
```

É exatamente esse tipo de rolagem que vocês vão usar hoje — um número sorteado decidindo o que acontece, através de um `if/elif/else` que já sabem escrever.

EXERCÍCIOS

Montando o jogo, peça por peça

Cada exercício soma sobre o anterior. As classes usadas são as que vocês já têm — `atacar()`, `defender()`, `usar_pocao()`, `receber_dano()` continuam exatamente como estão.

Exercício 1

ESCOLHER PERSONAGEM

No início do jogo, pergunte qual personagem o jogador quer usar, com `input()`. De acordo com a escolha (1 ou 2), crie o objeto correspondente. Se a pessoa digitar qualquer outra coisa, avise que a escolha é inválida e crie o primeiro personagem por padrão, pra o jogo não travar.

```
escolha = int(input("Escolha seu personagem (1 ou 2): "))

if escolha == 1:
    jogador = _____
elif escolha == 2:
    jogador = _____
else:
    print("Escolha invalida, comecando com o personagem 1.")
    jogador = _____
```

REQUISITOS MÍNIMOS

- `input()` convertido com `int()` antes de comparar
- `if/elif/else` cobrindo as duas opções válidas e um caso padrão
- `jogador` é sempre criado, não importa o que foi digitado

Exercício 2

TURNO DO JOGADOR

A cada turno, pergunte o que o jogador quer fazer — "atacar", "defender" ou "pocao" — com `input()`, sem converter. Cada resposta chama o método correspondente que já existe na classe do jogador.

```
acao = input("O que voce quer fazer? (atacar/defender/pocao): ")

if acao == "atacar":
    jogador.atacar(inimigo)
elif acao == "defender":
    _____
elif acao == "pocao":
    _____
else:
    print("Acao invalida — o turno passa sem fazer nada.")
```

REQUISITOS MÍNIMOS

- `input()` usado sem `int()` — a resposta é comparada como texto
- As três ações chamam os métodos que já existem — nenhum método novo é escrito aqui
- Uma resposta inválida não quebra o jogo, só não faz nada nesse turno

Exercício 3

DANO VARIÁVEL

Voltem no método `atacar()` que vocês já escreveram, e troquem o dano fixo (`self.ataque`) por um valor sorteado com `random.randint()`, entre `self.ataque - 3` e `self.ataque + 3`. É a primeira vez mudando um método que já existia, em vez de escrever um novo.

```
def atacar(self, alvo):
    dano = random.randint(_____, _____)
    print(self.nome, "ataca", alvo.nome + "!")
    alvo.receber_dano(dano)
```

REQUISITOS MÍNIMOS

- `import random` no topo do arquivo
- `random.randint()` usando `self.ataque - 3` e `self.ataque + 3` como limites
- O valor sorteado é o que é passado pra `receber_dano()`, não o `self.ataque` original

Exercício 4

CHANCE DE ACERTO · D20

Ainda dentro de `atacar()`, antes de calcular o dano, sorteiem um d20 (`random.randint(1, 20)`). Se o resultado for menor que 8, o ataque erra — nenhum dano é aplicado, só uma mensagem avisando. Se for 8 ou mais, o ataque acontece normalmente, com o dano sorteado do Exercício 3.

```
def atacar(self, alvo):
    rolagem = random.randint(1, 20)

    if rolagem < 8:
        print(self.nome, "errou o golpe!")
    else:
        dano = random.randint(self.ataque - 3, self.ataque + 3)
        print(self.nome, "ataca", alvo.nome + "!")
```

REQUISITOS MÍNIMOS

- `random.randint(1, 20)` sorteado antes de qualquer coisa
- `if/else` separando o caso de erro do caso de acerto
- No caso de erro, `receber_dano()` não é chamado de jeito nenhum

JUNTANDO TUDO

Da peça pro jogo completo

Os dois últimos não ensinam nada novo — só encaixam o que já foi construído.

Exercício 5

O LAÇO COMPLETO

Juntem os Exercícios 1 e 2 num só `while`: enquanto o jogador **e** o inimigo continuarem vivos, o turno do jogador roda, e se o inimigo ainda estiver vivo depois disso, o inimigo também ataca (usando o mesmo `atacar()`, já com dano variável e chance de erro dos Exercícios 3 e 4). Quando o laço terminar, uma mensagem final diz quem venceu.

```
while jogador.vida > 0 and inimigo.vida > 0:
    print("--- Novo turno ---")

    # turno do jogador (exercício 2)

    if inimigo.vida > 0:
        inimigo.atacar(jogador)

    if jogador.vida > 0:
        print(jogador.nome, "venceu o jogo!")
    else:
        print(inimigo.nome, "venceu o jogo!")
```

REQUISITOS MÍNIMOS

- `while` com `and`, checando os dois lados
- O turno do inimigo só acontece se ele continuar vivo depois do turno do jogador
- O jogo termina sozinho, com uma mensagem dizendo quem venceu

Exercício 6 DESAFIO

VÁRIOS INIMIGOS EM SEQUÊNCIA

Troquem o inimigo único por uma lista de inimigos. Depois de vencer um, o próximo da lista entra na batalha automaticamente, reaproveitando o mesmo laço do Exercício 5 — até a lista acabar (vitória final) ou o jogador perder antes disso (derrota).

REQUISITOS MÍNIMOS

- Uma lista de inimigos, não um único objeto
- O laço do Exercício 5 roda de novo pra cada inimigo da lista, sem copiar e colar o código
- Poções usadas contra um inimigo não voltam pro próximo — `self.pocoas` continua contando do jogo inteiro, não reseta a cada batalha

Sem gabarito pra esse conjunto — do Exercício 5 em diante, o jogo já roda sozinho, e é o primeiro momento em que dá pra realmente jogar do começo ao fim.